





# Optimizing Graph I/O for In-memory Computations[1]

Ali Brooks and **George M. Slota**
*Rennselaer Polytechnic Institute*

IEEE High Performance Extreme Computing Conference (HPEC)

September 15th, 2026

[1]This work is supported by the National Science Foundation under Grant No. 2047821.

# One big caveat before we begin

**Caveat**: The wealth of graph data, storage formats, frameworks, I/O and preprocessing methods make a fully comprehensive discussion and fair experimental evaluation nearly impossible. Apologies in advance.

# A problem with graph computations...

There is no single accepted on-disk graph format. Graph data preprocessing for framework-specific formats and I/O costs are often not a target for optimizations, despite possibly **requiring the significant majority of end-to-end time** for basic graph computations.

Outside of work involving out-of-core processing, these considerations have been relatively ignored in the literature.

# How do we generally store and read graphs?

**Observation**: The most popular graph data repositories (SNAP, SuiteSparse, Koblenz, etc.) generally offer graphs downloaded in a human-readable text edge list format (i.e., *.mtx or similar).

Graph processing frameworks then do one of two things.

1. Directly read that edge list and construct an in-memory graph representation, usually adjacency-based (e.g., CSR).
2. First convert that edge list to a adjacency-based text or binary format on-disk to accelerate I/O and graph construction over multiple runs.

**However, a focus on optimizations for both (1) and (2) is often absent in practice.**

# For example, some popular frameworks:

Running a single node with 2x CPUs (top) and 4x GPUs (bottom)

Running on com-friendster, assuming an initial text-based edge list file downloaded from a popular repository. The time from initial file to the start of actual computation can be substantial.

| | | Timings (s) | | | | |
|---|---|---|---|---|---|---|
| Framework | Format | PreProc | I/O | Graph | Total | Disk (GB) |
| CombBLAS | Edge-Text | **0.0** | 337 | 259 | 596 | 29.0 |
| Ligra | Adj-Text | 1378 | 26.0 | **0.02** | 1404 | 29.6 |
| Ligra | Adj-Bin | 1644 | 15.7 | 0.09 | 1660 | 13.9 |
| Galois-CPU | Adj-Bin | 410 | 97 | 7.89 | 1414 | 13.9 |
| This Work | Edge-Bin | 21.5 | **5.68** | 9.30 | **36.5** | **13.5** |
| CuGraph | Edge-Text | **0.0** | 69.4 | 19.5 | 88.9 | 29.0 |
| Galois-GPU | Adj-Bin | 249 | 297 | 9.53 | 743 | 13.9 |
| This Work | Edge-Bin | 21.5 | **32.5** | **0.96** | **55.0** | **13.5** |

**This work seeks to address this common bottleneck.**

# Our Contributions

- We study the graph processing bottleneck in the context of I/O and preprocessing.
- We design and optimize parallel algorithms for CPU, GPU, and distributed environments for preprocessing, I/O, and construction of an in-memory graph CSR-based representation.
- Our preprocessing and I/O methods run at nearly the speed of the filesystem.
- We can read and construct a distributed CSR (plus transpose) for the 128 billion edge 2012 Web Data Commons graph **in 40 seconds across 512 GPUs**.

# Data File to Graph Processing

Our optimized pipeline from downloaded text-based edge list to actual graph processing. Let's discuss the main ideas.

**Algorithm 1** Data File to Graph Processing

1: $F_b \leftarrow$ Preprocess($F_t$) ▷ Preprocessing of text to binary
2: **Begin Graph Program**
3: $E \leftarrow$ ReadBinaryEdgeList($F_b$)
4: $E' \leftarrow$ DistributeEdges($E$) ▷ If in distributed memory
5: $G \leftarrow$ ConstructGraph($E'$)
6: PerformGraphComputation($G$)

**Overall Idea:** We use a *binary edge list* format, as it offers an excellent tradeoff between storage, preprocessing, I/O, and graph construction costs relative to other formats.

Binary edge list: simply an edge list as $(u, v)$ edge tuples converted from a downloaded text list to a binary format.

# Text to Binary Edge List Preprocessing

**Algorithm 2** Edge Binary Preprocessing

```
1:  procedure PREPROCESS(Text Edge List File F_t)
2:      Begin Parallel
3:      r_s, r_e ← GetMyReadOffsets(F_t)              ▷ Start/end file offsets
4:      m ← CountEdges(F_t, [t_s, t_e])
5:      M ← Reduce(m)                                 ▷ Get global number of edges
6:      Single Thread: F_b ← AllocateOutputFile(M)
7:      for r_c = r_s to r_e by BUFFER_SIZE do
8:          t_buff ← ReadEdges(F, t_c, BUFFER_SIZE)
9:          t_o ← GetWriteOffset(O, r_c)
10:         F_b[t_o] ← WriteEdges(t_buff)
11:     End Parallel
```

**Idea 1:** Use optimized functions to process at speed-of-disk (e.g., `mmap`, `memchr`, `posix_fallocate`, `pwrite`).

**Idea 2:** Use end-to-end parallelization to take advantage of parallel filesystems (when possible).

# Distributed Memory Edge Exchange

After reading in our binary edge list, if we're in distributed memory

**Note:** We first read from our binary edge list into memory across ranks using the same parallelization strategy.

**Algorithm 3** Distributed Memory Edge Exchange

1: **procedure** DISTRIBUTEEDGES(Edge List $E$)
2: $Counts_{Send} \leftarrow$ CountEdgesPerRank($E$)
3: $Buff_{Send} \leftarrow$ ConstructSendBuffer($E, Counts_{Send}$)
4: $Counts_{Recv} \leftarrow$ ExchangeSendCounts($Counts_{Send}$)
5: $E' \leftarrow$ AllocateReceiveBuffer($Counts_{Recv}$)
6: $E' \leftarrow$ ExchangeSendBuffers($E, Counts_{Send}, Counts_{Recv}$)

**Main Idea:** Every computational aspect can utilize $O(1)$ depth parallel routines. Minimize overheads and let the thousands/millions of threads do what they need to do. Communication costs *should* dominate.

# Graph Construction using a CSR-like Format

**Algorithm 4** Graph Construction using a CSR-like Format

1: **procedure** ConstructGraph(Edge List $E'$)
2: $Offsets \leftarrow$ CountDegrees($E'$)
3: $Offsets \leftarrow$ PrefixSums($Offsets$) ▷ CSR Offsets array
4: $Offsets_{copy} \leftarrow$ Copy($Offsets$)
5: $Adjacencies \leftarrow$ ConstructAdjList($E', Offsets_{copy}$)
6: $Adjacencies \leftarrow$ SortAdjacencies($Adjacencies, Offsets$)
7: $G \leftarrow \{Adjacencies, Offsets\}$

**Main Idea:** We can utilize $O(1)$ depth routines for everything except the $O(\log n)$ prefix sums and sorting. An efficient implementation should take advantage of this parallelism.

# Experimental Setup

## Test graphs and test system

| Name | Vertices | Edges | Description | Abbr. |
|---|---|---|---|---|
| USA-road-d | 24 M | 116 M | Road Network | USA |
| twitter-2010 | 41 M | 2.9 B | Follower Network | TW |
| com-friendster | 65 M | 3.6 B | Social network | FR |
| MOLIERE_2016 | 30 M | 6.7 B | Hypothesis Network | MO |
| web-ClueWeb09 | 1.7 B | 16 B | Web Crawl | CW |
| gsh-2015 | 1.0 B | 66 B | Web Crawl | GSH |
| WDC12 | 3.6 B | 257 B | Web Crawl | WDC12 |

`AiMOS` (multi-node runs): 134 POWER9s and 512 V100s on 67 nodes. Parallel filesystem listed at 80 GB/s.

`Zepy` (single node GPU runs): 28-cores and 4x A100s. Single disk filesystem benchmarked at 0.52 GB/s sequential parallel read.

`Bella` (single node CPU runs): 128-cores with RAID 6 filesystem benchmarked at 3.9 GB/s sequential parallel read.

# Experimental Setup

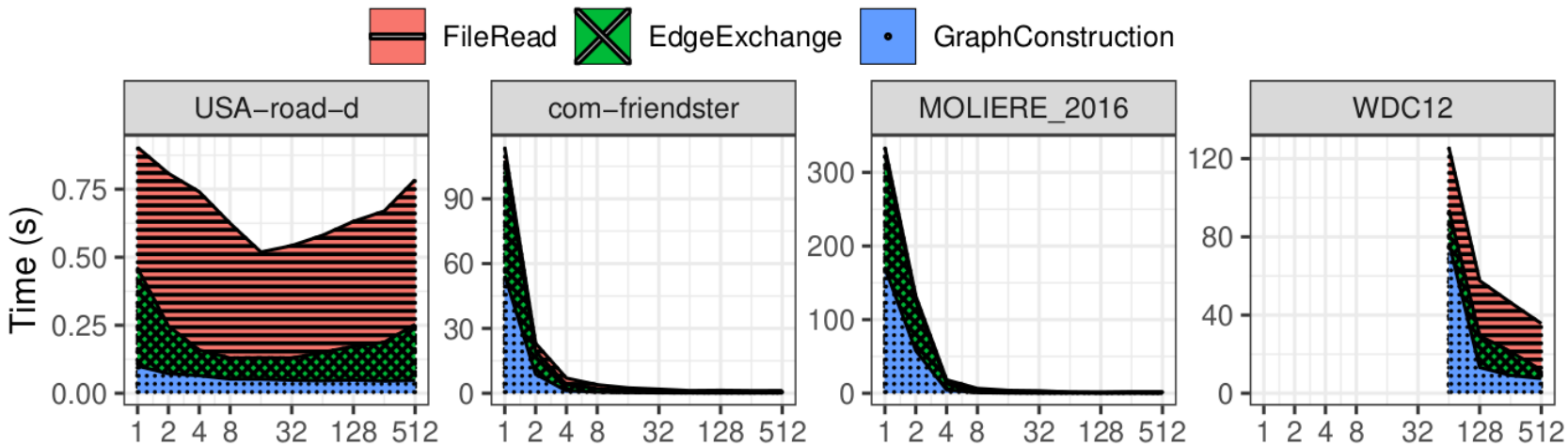
## Framework comparisons

| Framework | Context | Format | Par. I/O | Sorted |
|---|---|---|---|---|
| Ligra | Shared CPU | Text/Bin. adj. | N | N |
| CuGraph | Dist. GPU | Text edge | Y | Y |
| CombBLAS | Dist. CPU | Text edge | N | Y |
| Galois (Gluon) | All | Binary adj. | Y | Y |
| This Work | All | Binary edge | Y | Y |

The experimental graph frameworks with their computational contexts, graph storage format, whether parallelization of file I/O is supported (beyond MPI), and whether the adjacencies in the CSR-like graph structure are sorted during graph construction or file conversion (in these experiments).

# Strong Scalability

## Strong Scaling up to 512 GPUs

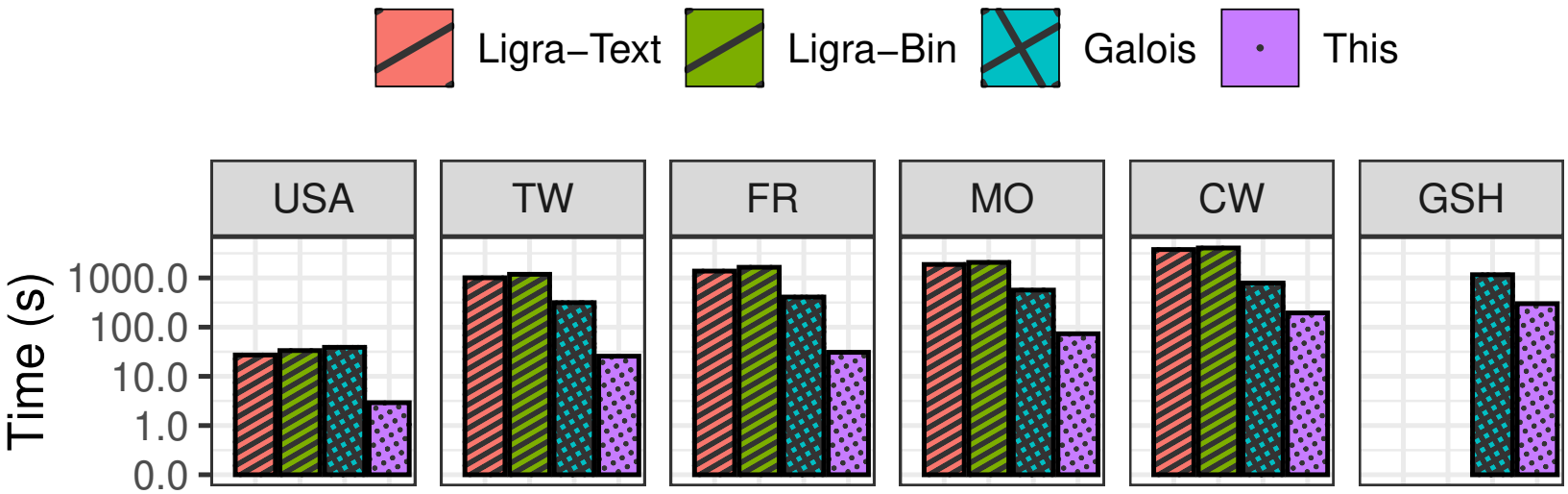



**All inputs strong scale to 512 GPUs except for the smallest road network, which can fit on a single GPU replicated about 20×.**

CPU results were similar but with a slower edge exchange (OpenMPI vs. NCCL) and graph construction phase (orders-of-magnitude fewer threads). Scaling slowed down around 128-256 ranks.

# Comparisons: Preprocessing

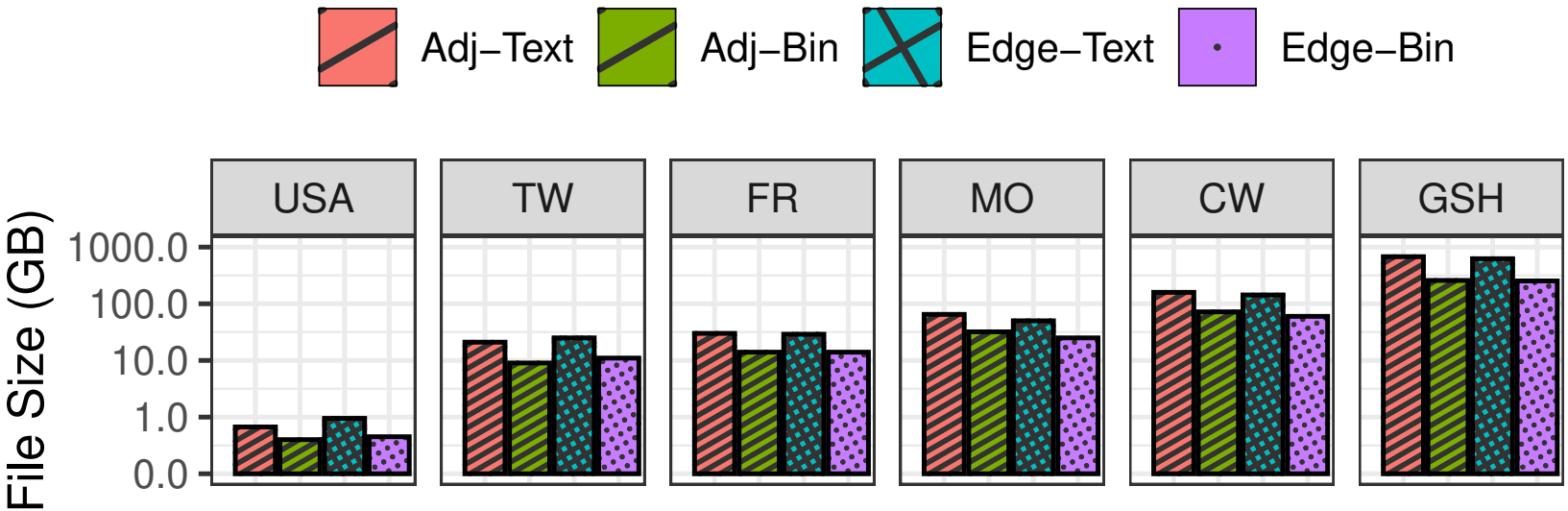
Converting from a text-based edge list format




Preprocessing times on `bella`, to convert from a text edge list to a binary edge list (This), binary adjacency (Ligra and Galois), and text adjacency (Ligra).

We are generally **1 to 3 orders-of-magnitude faster** and within a 2x factor of tested filesystem limits.

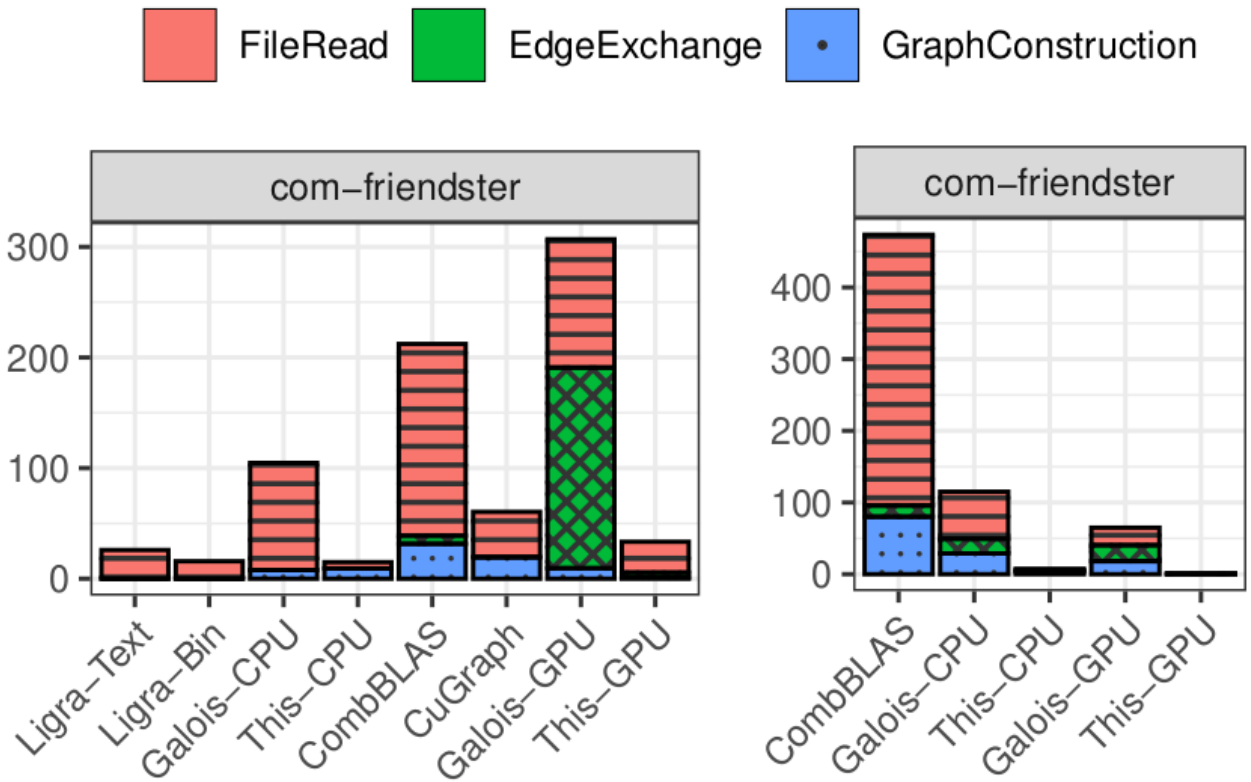
# Comparisons: Storage Cost




- Binary adjacency formats require $|V| + 2|E|$ for undirected graphs and directed graphs with transpose, $|V| + |E|$ without.
- Binary edge lists always require $2|E|$.
- Text formats are proportion to above, but with the cost of the $|E|$ term also increasing proportional to $|V|$.

# Comparisons: Timings on a single, multiple nodes

**Single Node:** 2× CPUs or 4× A100 GPUs (left)
**Multi Node:** 64 CPU ranks or 64 V100 GPUs (right)




Bespoke formats still need considerable processing in distributed memory. Unoptimized I/O dominates times.

# Our Conclusion: The case for binary edge lists

We presented an approach for optimizing the graph preprocessing and I/O pipeline using a binary edge list format.

**This format has many benefits in practice:**

- Preprocessing is faster than any other bespoke format.
- I/O is extremely fast and efficient to parallelize.
- Edge distribution is simpler and more easily generalizable than other methods, including both 1D and 2D distributions.
- Graph construction is not a bottleneck relative to direct binary adjacency-based formats.
- More space efficient than binary adjacency formats for undirected graphs or directed graphs with the transpose.

Contact: `gmslota@gmail.com`, `www.gmslota.com`